

同济大学计算机系

类 Rust 一遍式编译器实验报告



专 业 计算机科学与技术

组 员 _____

组 员 _____

组 员 _____

授课老师 卫志华、高珍老师

摘 要

本课程设计在既有类 Rust 语言词法、语法、语义分析与四元式生成器基础上，按照 2026 年课程设计附件完成了一体化改造。系统以手写递归下降语法分析器为主控，词法分析器不再预先扫描整个文件，而是通过 `Lexer.next_token()` 在语法分析需要时提供下一个记号；每解析完成一个函数声明，解析器立即触发语义动作生成对应 `IRFunction`，从而形成函数粒度的语法制导翻译。完整程序随后经过符号表、类型、可变性、借用、控制流和返回路径检查，只有语义门禁通过时才写出中间代码与目标代码文件。

目标代码后端将四元式降低为 GNU Intel 语法的 x86-64 Windows 汇编：标量与临时值分配栈槽，数组和元组采用连续存储，函数调用遵循前四参数寄存器与 32 字节 shadow space 规则，引用保存栈地址，数组访问在长度可知时插入运行时上下界检查。生成的 .s 文件可由 GCC 直接汇编链接为 .exe；综合示例覆盖分支、循环、数组、元组、借用、块表达式和函数调用，执行退出码为 42。

实现过程中还补齐了不可变变量首次初始化、可变借用互斥、缺失返回路径、数组/元组越界等语义规则，并修复 for 循环中 continue 跳过自增以及变量重影导致 IR 栈槽同名覆盖等深层缺陷。最终自动回归共 44 组检查全部通过，源码入口和 PyInstaller 单文件编译器均完成“源程序→IR→汇编→可执行程序”的端到端验证。

关键词：类 Rust；一遍编译；递归下降；语法制导翻译；四元式；x86-64；借用检查

目 录

一、课程设计概述.....	5
1.1 背景与目的.....	5
1.2 任务范围.....	5
1.3 开发与运行环境.....	5
1.4 交付物与验收目标.....	6
二、需求分析.....	7
2.1 可行性分析.....	7
2.2 功能需求.....	7
2.3 非功能需求.....	7
2.4 语法规则覆盖与优秀档策略.....	8
2.5 验收准则.....	8
三、总体方案设计.....	9
3.1 总体架构与数据流.....	9
3.2 对“一遍”的实现定义.....	9
3.3 模块职责与接口.....	10
3.4 核心数据结构.....	10
3.5 错误模型与输出策略.....	10
四、前端与语法制导翻译详细设计.....	11
4.1 按需词法分析.....	11
4.2 递归下降与有限前看.....	11
4.3 表达式优先级与左递归消除.....	11
4.4 函数归约点的语义动作.....	12
4.5 声明、赋值和函数调用翻译.....	12
4.6 控制流翻译与回填.....	12
4.7 块、if 与 loop 表达式.....	13
五、静态语义与复合类型.....	14
5.1 函数表、作用域栈与初始化.....	14
5.2 类型推导、函数调用和返回路径.....	14
5.3 不可变性与借用模型.....	14
5.4 数组、元组与边界检查.....	14
5.5 错误定位示例.....	15

目 录 (续)

六、中间代码与目标代码.....	16
6.1 四元式模型与指令集.....	16
6.2 典型翻译示例.....	16
6.3 IR 与汇编文件格式.....	16
6.4 x86-64 栈帧与 Windows ABI.....	17
6.5 主要指令映射.....	17
6.6 汇编、链接与可执行输出.....	17
6.7 目标后端边界.....	18
七、程序实现与使用.....	19
7.1 项目目录.....	19
7.2 关键类与函数索引.....	19
7.3 命令行参数.....	20
7.4 独立编译器打包.....	20
7.5 运行界面与结果.....	20
八、测试设计与实验结果.....	21
8.1 分层测试策略.....	21
8.2 44 组回归结果.....	21
8.3 综合样例覆盖.....	22
8.4 关键缺陷的回归证据.....	22
8.5 源码入口与打包程序交叉验证.....	22
8.6 测试局限.....	22
九、问题解决与 AI 辅助历程.....	23
9.1 从分离流水线改为 Parser 主控.....	23
9.2 控制流与重影缺陷.....	23
9.3 借用、返回与边界规则补强.....	23
9.4 目标后端中的工程问题.....	23
9.5 AI 工具使用披露.....	23
9.6 对 AI 辅助的反思.....	24
十、总结与展望.....	25
10.1 工作总结.....	25
10.2 后续工作.....	25
10.3 设计体会.....	25
参考资料.....	26
附录 A: 核心文法摘要.....	27
附录 B: 复现实验命令.....	28

一、课程设计概述

1.1 背景与目的

编译器把高级语言源程序逐步转换为机器可执行形式。课程设计要求不仅理解词法、语法、符号表和中间代码等独立概念，还要把这些概念组织成可运行、可验证、可解释的完整系统。类 Rust 语言同时包含常见的函数、变量、表达式与控制流，又引入 mut、引用、数组、元组和表达式块等语义约束，适合作为编译原理综合训练对象。

本项目的首要目标是把之前相互分离的词法分析和语法分析改造成语法主控的一遍式 workflow；第二个目标是在语法分析过程中以函数为单位执行中间代码语义动作；第三个目标是补足目标代码后端，使输出不止停留在四元式，而能进一步形成可由 GCC 汇编链接的 x86-64 程序；最后通过自动回归和综合程序执行证明各阶段连接正确。

1.2 任务范围

- 输入 UTF-8 编码的类 Rust 源文件，按需完成词法识别并提供包含行列信息的 Token。
- 采用递归下降方法构造 AST，支持附件 0.1–9.3 中的函数、声明、表达式、分支、循环、引用、数组、元组和表达式块。
- 建立函数表与块级作用域栈，执行类型推导、初始化、可变性、借用、调用、返回路径和边界检查。
- 在函数归约点生成四元式 IR，语义检查通过后写入 .ir 文件。
- 把 IR 降低为 x86-64 GNU Intel 汇编，写入 .s 文件，并可调用 GCC 生成 .exe。
- 提供命令行界面、独立 Windows 编译器、综合样例、错误定位和自动回归测试。

1.3 开发与运行环境

项目	配置/版本	用途
实现语言	Python 3.10+（验证环境含 3.13）	编译器前端、IR、汇编后端和测试驱动
汇编链接	MSYS2 GCC 15.1	将 GNU Intel x86-64 .s 链接为 Windows .exe
独立打包	PyInstaller	生成 dist/rust_like_compiler.exe
办公与验收	Word/LibreOffice、PowerPoint	设计报告与答辩材料的渲染检查
运行平台	Windows x86-64 / PowerShell	CLI、GCC、退出码验证

1.4 交付物与验收目标

课程附件要求提交源程序、编译器可执行程序和设计说明书。本工作区对应交付为 analyzer 源码目录、dist/rust_like_compiler.exe、完整 README、本实验报告和汇报 PPT。验收采用可复现命令而不是主观截图：运行回归程序必须全部通过；综合程序必须生成 .ir、.s、.exe；目标程序退出码必须等于 42；编译错误必须返回非零状态并带行列定位。

二、需求分析

2.1 可行性分析

技术上，既有项目已经具备 Token、AST、语义检查和四元式结构，重构的主要风险集中在扫描接口、语法制导时机和目标代码映射。按需 Lexer 可以在不改变文法函数结构的前提下替换预扫描 Token 列表；函数级 IR 生成不依赖全局类型信息，适合在 FunctionDecl 解析完成后执行；x86-64 后端以已有四元式为输入，可以独立实现和测试。因此本项目无需推倒重写，符合最小改造原则。

工程上，Python 标准库即可完成主体实现，GCC 与 PyInstaller 只用于可选汇编链接和独立打包。测试方面，原有示例可作为回归基线，并可增加内存中的专项用例验证借用、越界和返回路径。时间上，各模块接口明确，可以按“扫描接口→语法回调→语义补强→后端→端到端测试”的顺序分阶段验收。

2.2 功能需求

编号	需求	可验证结果
FR-01	语法分析按需调用词法子程序	Parser 持有 Lexer; _ensure_token 才调用 next_token
FR-02	一遍顺序扫描源程序	不预先 tokenize; 局部回溯仅复用已缓存 Token
FR-03	语法制导生成中间代码	每完成 FunctionDecl 触发 on_function 回调
FR-04	生成并保存 IR	语义通过后写出同名 .ir 文件
FR-05	生成可汇编目标代码	写出 .s; GCC 生成并运行 .exe
FR-06	覆盖类 Rust 文法与语义	规则 0.1–9.3 均有实现与测试证据
FR-07	提供错误诊断	词法/语法含行列、源码行和插入符; 语义错误分类清晰
FR-08	提供编译器可执行程序	PyInstaller 单文件程序使用同一 CLI 参数

2.3 非功能需求

- 正确性：合法程序各阶段输出稳定；非法程序不得写出可误用的最终产物。

- 可解释性：Parser、SemanticAnalyzer、CodeGenerator、AssemblyGenerator 之间有清晰职责边界。
- 可测试性：阶段结果均可独立检查，CLI 返回码可被脚本消费。
- 可移植性：不生成汇编时无第三方 Python 依赖；汇编后端明确限定 Windows x86-64。
- 可维护性：改造保持已有 AST 和 IR 数据模型，避免为课程范围引入不必要框架。

2.4 文法规则覆盖与优秀档策略

规则组	实现内容	状态
0.1–1.5	mut、i32、左值、多函数、空语句、参数、返回类型	完成
2.0–2.3	声明、延迟推断、赋值、声明初始化、重影	完成
3.1–4.3	优先级表达式、调用、if/else/else-if	完成
5.0–5.4	while、范围/数组 for、loop、break、continue	完成
6.1–6.4	不可变性、不可变/可变引用、借用与解引用	完成
7.0–7.4	块表达式、函数尾表达式、if/loop 表达式	完成
8.1–8.3	定长数组、字面量、迭代、下标读写与越界	完成
9.1–9.3	元组类型、表达式、数字字段读写与越界	完成
目标代码	x86-64 汇编、GCC 链接、可执行程序	完成

按附件星级分组，5.x、6.x、7.x、8.x 和 9.x 扩展均已覆盖，远高于申请优秀所需的 7 星门槛。附件评分页写有 7.1–7.5，但正文仅定义到 7.4；本实现以正文实际规则 7.0–7.4 为准，并在报告中保留这一源稿疑点。

2.5 验收准则

1. python -m compileall -q src run.py run_regression.py 返回 0。
2. python run_regression.py 输出 44 个 [PASS]，末尾为 Regression passed。
3. 综合样例 course_full.rs 生成 course_full.ir、course_full.s 和 course_full.exe。
4. 运行 course_full.exe 后 PowerShell 的 \$LASTEXITCODE 等于 42。
5. dist/rust_like_compiler.exe 能独立完成相同编译链路。

三、总体方案设计

3.1 总体架构与数据流

系统采用“语法主控、函数级翻译、程序级语义门禁、后端独立降低”的结构。Parser 仅在当前 Token 不足时向 Lexer 请求下一个记号；FunctionDecl 完成后立即交给 CodeGenerator 生成 IRFunction；全部函数构成 Program AST 后，SemanticAnalyzer 先收集签名再逐函数检查。只有语义通过，IRProgram 才进入文件输出和目标后端。

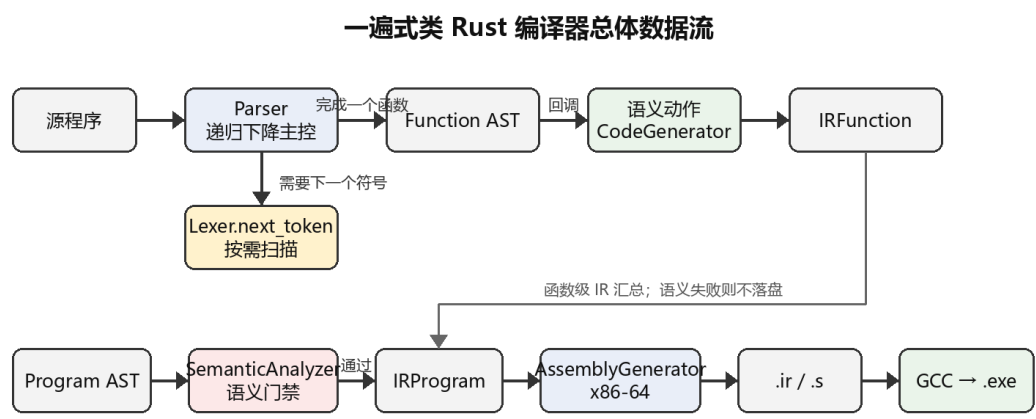


图 3-1 一遍式编译器总体数据流

3.2 对“一遍”的实现定义

本项目中的“一遍”指词法和语法只按源文件方向顺序推进一次，不再先独立生成完整 Token 文件、再启动第二套扫描。Parser 为了区分“块末尾表达式”和“表达式语句”会进行有限局部试探；试探失败时只回退 Token 索引，已经扫描的字符与 Token 保留在缓存中，不会让 Lexer 回退或重新扫描源字符串。因此源字符流仍只被词法器消费一次。

中间代码的语义动作位于函数声明归约点，而不是等整个 AST 完成后统一遍历。考虑到课程文法允许调用后定义函数，静态语义仍保留程序级签名收集与校验阶段。这一边界既满足“语法分析同时生成 IR”的要求，也避免为了形式上的单阶段而牺牲前向调用和清晰错误诊断。

3.3 模块职责与接口

模块	输入	输出	核心职责
lexer.py	源码字符	Token	空白/注释跳过、最长匹配、行列维护、next_token
parser.py	Lexer/Token	AST	递归下降、优先级、有限前看、函数完成回调
semantic.py	Program AST	通过/错误	函数表、作用域、类型、借用、边界、返回路径
codegen.py	FunctionDecl	IRFunction	语法结构到四元式、临时变量、标签和回填
target.py	IRProgram	x86-64 文本	栈布局、ABI、指令选择、边界检查
compiler.py	源码	CompilationResult	协调各阶段、文件写出和 GCC 调用
run.py/main.py	CLI 参数	终端/文件	用户入口、显示开关、退出码
run_regression.py	样例/内存程序	PASS/FAIL	44 组回归与端到端执行

3.4 核心数据结构

- Token(kind, lexeme, line, column): 保留类别、原词素与定位信息。
- AST: Program、FunctionDecl、Block、语句节点、表达式节点和 TypeNode 层次。
- VariableSymbol: 记录类型、mut、初始化状态和当前借用计数；作用域使用字典栈。
- Quad(op, arg1, arg2, result): 统一表达算术、控制流、调用、引用和复合类型操作。
- IRFunction/IRProgram: 保存函数参数与四元式序列，作为目标后端输入。
- FrameLayout: 保存 IR 名称到 rbp 偏移、复合对象 backing 区域、长度和总帧大小。

3.5 错误模型与输出策略

LexerError、ParserError、SemanticError、CodeGenerationError 与 TargetCodeError 均继承 CompilerError。run_file 捕获编译阶段错误、文件错误和 GCC 子进程错误，成功返回 0，失败返回 1。词法与语法错误保留源代码行并用插入符标出列位置；语义与后端错误给出违反的规则和对象名称。

输出采用事务式门禁：Compiler.compile 先完成 AST、函数级 IR、语义检查和汇编文本构造，write_outputs 只在整个调用成功后创建目录并写出 .ir/s；--assemble 在文件写出后调用 GCC。这样语义失败不会留下看似成功的最终目标文件。

四、前端与语法制导翻译详细设计

4.1 按需词法分析

Lexer 保存 `source`、`index`、`line`、`column`、`emitted_tokens` 和 `finished` 状态。`next_token` 每次跳过空白与注释后只识别一个 `Token`：标识符先完整读取再查关键字表；数字连续读取十进制字符；双字符运算符 `->`、`==`、`!=`、`>=`、`<=`、`..` 优先于单字符符号，符合最长匹配原则；非法字符和未闭合块注释立即抛出 `LexerError`。

```
def _ensure_token(self, index: int) -> None:
    while self.token_source is not None and len(self.tokens) <= index:
        token = self.token_source.next_token()
        self.tokens.append(token)
        if token.kind == TokenKind.EOF:
            break
```

Parser 的 `current`、`check` 和 `lookahead` 最终都通过 `_ensure_token` 请求 `Token`。若构造 Parser 时传入的是旧式 `Token` 列表，接口仍可用于回归测试；正式编译入口始终传入 Lexer 实例。到达 EOF 后 Lexer 重复返回同一个 EOF，避免越界并保证终止。

4.2 递归下降与有限前看

每个主要非终结符对应一个解析函数：`parse_program` 读取函数串，`_parse_function_decl` 读取函数头和块，`_parse_statement` 根据首符选择声明、`return`、`if`、循环或表达式语句，`_parse_type` 递归解析引用、数组和元组类型。`expect` 负责消费预期 `Token`，失败时结合当前位置构造 `ParserError`。

块文法允许最后一个无分号表达式作为块值。解析器先记住 `pos`，尝试解析表达式；若其后紧接右花括号，就把表达式存入 `Block.result`；否则恢复 `pos` 再按普通语句解析。由于恢复的是 `Token` 索引而不是 Lexer 字符位置，前看不会造成字符流二次扫描。

4.3 表达式优先级与左递归消除

附件表达式产生式包含左递归，不能直接翻译为递归下降函数。实现把比较、加减、乘除分别改写为“较高优先级子表达式 + 零个或多个运算后缀”，在 `while` 循环中构造左结合 `BinaryExpr`。后缀层再处理调用、数组下标和元组字段，因此 `a + b * c[0]` 会先完成下标与乘法，再形成加法根节点。

```

comparison ::= additive (compare_op additive)*
additive   ::= term (('+' | '-') term)*
term       ::= postfix (('*' | '/') postfix)*
postfix    ::= primary (call | '[' expression ']' | '.' NUMBER)*

```

4.4 函数归约点的语义动作

Compiler 创建 Lexer、Parser 和 CodeGenerator，向 parse_program 传入 on_function 回调。每次 _parse_function_decl 返回 FunctionDecl 后，解析器立即调用回调；CodeGenerator.generate_function 为该函数建立 IRBuilder、作用域重命名器和循环标签栈，产生 IRFunction。最终 Program AST 与 IRFunction 列表同时得到。

```

ast = parser.parse_program(
    on_function=lambda function: ir_functions.append(
        code_generator.generate_function(function)
    )
)
SemanticAnalyzer().analyze(ast)
ir = IRProgram(functions=ir_functions)

```

4.5 声明、赋值和函数调用翻译

声明产生 decl 四元式；带初始化声明在 decl 后生成 =；普通赋值先计算右侧，再根据左值种类生成 =、*=、[]= 或 .=。函数调用先从左到右计算实参并生成 param，再产生 call(callee, argc, result)。没有返回值的调用结果为占位符，语义阶段禁止其参与运算。

重影是本轮修复的重要问题。AST 允许同一作用域再次 let 同名变量，但若 IR 仍用源变量名，后一个绑定会覆盖前一个栈槽。CodeGenerator 现在为每次声明分配唯一内部名，例如 v1_value、v2_value，并在作用域栈中维护源名到内部名的映射；查找从栈顶向下进行，离开块时映射自动失效。

4.6 控制流翻译与回填

if/else-if/else 为每个条件产生 jz 到下一分支标签，并在执行完命中分支后 j 到公共结束标签。while 结构为 condition、end 两个标签；loop 为 begin、end；for 范围循环被降低为初始化、condition、body、increment、end。所有跳转目标在生成时已由 IRBuilder.new_label 分配，无需机器地址回填。

原实现中 for 循环的 continue 直接跳回条件标签，导致迭代变量不自增，综合程序会陷入死循环。本次改为在 loop_stack 中记录专用 continue_label，并让范围 for 的 continue 指向 increment 标签；自增完成后再跳回 condition。这一修复由专项四元式断言和综合可执行程序共同验证。

4.7 块、if 与 loop 表达式

`BlockExpr` 先生成内部语句，再返回 `Block.result` 的值；`IfExpr` 为各分支生成同一个结果临时变量，并要求存在 `else` 以保证值定义；`LoopExpr` 通过 `break expression` 把值写入结果临时变量后跳转到结束标签。语义分析保证多个可达 `break` 的值类型一致，普通 `loop` 语句则忽略返回值。

五、静态语义与复合类型

5.1 函数表、作用域栈与初始化

`SemanticAnalyzer` 先遍历 `Program` 收集所有函数签名，检查重名并记录形参类型和返回类型；随后逐函数压入函数作用域，把形参标记为已初始化，再递归分析函数体。每进入一个 `Block` 压入新字典，离开时释放该层变量与借用状态。变量查找从栈顶向下，天然支持块级重影。

声明没有类型也没有初始化时，变量保持类型未知和未初始化；首次赋值可以推导类型，即使变量未标注 `mut` 也允许完成一次初始化。完成初始化后，不可变变量不得再次赋值。右值读取要求变量已声明、类型已知且已初始化。

5.2 类型推导、函数调用和返回路径

`_type_of_expr` 自底向上计算类型。算术运算要求两侧为 `i32`，比较运算要求类型兼容并产生内部 `BoolType`；`if/while` 条件接受比较结果或整数真值；范围端点和数组下标必须为 `i32`；数组字面量元素类型一致，元组按位置保存异构类型。调用检查函数存在、实参数量相等、各位置类型一致，并返回签名中的返回类型。

返回检查包含单条 `return` 和控制流路径两层。`return value` 必须与函数声明一致；声明 `i32` 返回的函数还必须保证所有可达路径返回值，或以类型一致的块尾表达式结束。`if` 语句只有在 `then`、所有 `else-if` 和 `else` 都返回时才视为完整返回，`while/for` 默认不能保证执行。`missing_return_path` 专项用例验证了这一规则。

5.3 不可变性与借用模型

变量符号记录 `is_mut`、`immutable_borrows` 和 `mutable_borrow`。`&x` 增加不可变借用计数，允许多个不可变借用并存；`&mut x` 仅允许作用于可变左值，且要求当前没有任何借用；可变借用存在时禁止直接读写原变量。不可变引用解引用后不可写，可变引用解引用后可写。引用变量离开词法作用域时释放相应借用。

该模型有意采用适合课程设计讲解的词法作用域生命周期，没有实现 `Rust` 编译器的非词法生命周期、移动语义和生命周期参数。限制被显式写入 `README` 和本报告，避免把教学检查器描述为完整 `Rust` 借用检查器。

5.4 数组、元组与边界检查

`ArrayType` 保存元素类型和定长，数组表达式必须非空且元素类型一致；显式数组类

型还要求元素数量匹配。IndexExpr 的集合必须为数组且索引为 i32；常量索引在语义阶段直接验证 $0 \leq \text{index} < \text{len}$ 。TupleType 保存元素类型列表，FieldExpr 的字段必须为整数字面量且在范围内。数组/元组元素的可写性继承根变量的可变性。

动态数组索引无法在编译期确定时，目标后端根据 FrameLayout 中传播的长度插入 cmp/jl/jge；越界跳转到函数专用 bounds_error 标签并执行 ud2。元组字段由语法限定为字面量，已在语义阶段静态检查，因此目标层只需按 field*8 访问。

5.5 错误定位示例

```
3:5 变量声明语句末尾缺少 ';'，当前符号: '}'  
  }  
  ^
```

词法与语法错误包含行、列、源代码行和插入符。语义错误侧重规则说明，例如“不可变变量不能再次赋值”“可变借用不能与其他借用共存”“函数存在并非所有路径返回值”。这种分层信息便于在答辩时说明错误属于哪一编译阶段。

六、中间代码与目标代码

6.1 四元式模型与指令集

四元式统一为 `Quad(op, arg1, arg2, result)`，缺失操作数在文本中显示为下划线。
`IRBuilder` 负责临时变量 `t1`、`t2...`和标签 `L1`、`L2...`的唯一编号；`IRFunction` 保存函数名、内部参数名与四元式列表；`IRProgram` 按函数组织输出。

类别	操作	语义
声明/赋值	<code>decl, =</code>	分配逻辑位置；把 <code>arg1</code> 写入 <code>result</code>
算术/比较	<code>+ - * / < <= > >= == !=</code>	计算两操作数并写临时结果
控制流	<code>label, j, jz</code>	定义标签、无条件跳转、零条件跳转
调用	<code>param, call, return</code>	准备实参、调用函数、返回值
引用	<code>ref, ref_mut, deref, *=</code>	取地址、间接读取和间接写入
复合类型	<code>[], []=, ., .=, len</code>	数组/元组读写与长度

6.2 典型翻译示例

```
源程序: a = 1 + 2 * 3;

(*, 2, 3, t1)
(+, 1, t1, t2)
(=, t2, _, v1_a)
```

乘法节点位于加法子树中，因此先产生 `t1`，再产生 `t2`，最后写入经过重影重命名的 `v1_a`。结构化控制流同样被显式标签和跳转线性化，这使目标后端无需理解 `AST`。

6.3 IR 与汇编文件格式

`write_outputs` 以源文件 `stem` 命名产物，在默认的“源文件所在目录/build”或 `--output-dir` 指定目录中写出 UTF-8 `.ir` 与 `.s`。IR 文件以 `Function name(params):` 开头并为每条四元式编号；汇编文件以 `.intel_syntax noprefix` 和 `.text` 开头，每个函数导出 `.globl` 符号。

6.4 x86-64 栈帧与 Windows ABI

AssemblyGenerator 首先扫描函数四元式建立 FrameLayout。每个参数、局部变量和临时值分配 8 字节栈槽；数组/元组临时对象另分配连续 backing 区域，并让逻辑变量保存首地址。帧大小向 16 字节对齐。函数序言 push rbp、mov rbp, rsp、sub rsp, frame_size，返回路径统一跳到 epilogue。

Windows x64 调用约定使用 rcx、rdx、r8、r9 传递前四个整数/地址实参，更多实参写到调用者栈；调用者始终预留 32 字节 shadow space 并保持 16 字节对齐。返回值位于 rax。教学类型 i32 在类型系统中仍是 32 位整数，但物理栈槽统一使用 QWORD，以同时容纳引用地址并简化后端。

6.5 主要指令映射

IR	x86-64 核心序列	说明
+ / - / *	mov + add/sub/imul	结果写回目标栈槽
/	cqo + idiv	有符号除法，商在 rax
比较	cmp + setcc + movzx	规范化为 0/1
j / jz	jmp / test+je	函数内标签加名前缀防冲突
ref/ref_mut	lea rax,[rbp-offset]	引用保存真实栈地址
deref/*=	mov [rax]	间接读写
[]/=	base + index*8	已知长度时先做上下界检查
call	shadow space + 参数寄存器 + call	返回值保存到结果槽

6.6 汇编、链接与可执行输出

```
python run.py examples/course_full.rs --no-tokens --no-ast --assemble
& examples/build/course_full.exe
$LASTEXITCODE
# 42
```

assemble 使用 subprocess.run([gcc, asm_path, -o, executable_path], check=True)。综合样例的 main 无参数并返回 i32，最终计算值为 42；目标程序执行后 PowerShell 退出码为 42，证明生成的控制流、调用、复合类型和引用路径在机器层贯通。

6.7 目标后端边界

当前后端未实现优化、调试信息、浮点数、字符串、结构体、堆分配和完整 Rust 所有权系统；仅面向 Windows x86-64 与 GNU 汇编器。动态数组长度传播只覆盖本课程 IR 中可追踪的声明与赋值。对课程范围外的四元式操作，后端抛出 `TargetCodeError`，而不是静默生成错误指令。

七、程序实现与使用

7.1 项目目录

analyzer/		
└ run.py	命令行入口	
└ run_regression.py	44 组自动回归	
└ README.md	完整使用与设计说明	
└ dist/rust_like_compiler.exe		
└ examples/course_full.rs	综合可执行样例	
└ src/		
└ compiler.py	lexer.py	parser.py ast_nodes.py
└ semantic.py	ir.py	codegen.py
└ target.py	main.py	errors.py

7.2 关键类与函数索引

符号	文件	职责
Lexer.next_token	src/lexer.py	按需扫描一个 Token
Parser.parse_program	src/parser.py	函数串解析与 on_function 回调
Parser._ensure_token	src/parser.py	请求 Token 并维护前看缓存
SemanticAnalyzer.analyze	src/semantic.py	程序级签名与逐函数语义检查
CodeGenerator.generate_function	src/codegen.py	FunctionDecl 到 IRFunction
AssemblyGenerator.generate	src/target.py	IRProgram 到 x86-64
Compiler.compile	src/compiler.py	统一编译协调器
run_file	src/main.py	CLI 展示、文件输出、GCC 与错误返回

7.3 命令行参数

参数	作用
input_file	输入类 Rust 源文件；缺省为 examples/course_full.rs
--no-tokens	不打印 Token 列表
--no-ast	不打印 AST JSON 和树
--no-semantic	跳过语义检查，仅用于调试
--no-ir	不在终端打印四元式
--show-asm	在终端打印 x86-64 汇编
--output-dir DIR	指定 .ir/.s/.exe 输出目录
--no-write	不写文件，不能与 --assemble 同用
--assemble	调用 GCC 生成可执行程序

7.4 独立编译器打包

dist/rust_like_compiler.exe 由 PyInstaller --onefile 生成，使用者无需安装 Python，参数与 python run.py 完全一致。打包程序不内置 GCC；若仅生成 .ir/.s 无需额外工具，若使用 --assemble 仍需系统 PATH 中存在 GCC。最终可执行编译器 SHA-256 为 7D202334255F8A2A6A830957C330E9A353C35FC324505187915193D8886888C5。

7.5 运行界面与结果

```
PS> .\dist\rust_like_compiler.exe .\examples\course_full.rs `
    --no-tokens --no-ast --no-ir --output-dir D:\TEMP\rust-demo --assemble
Semantic analysis passed.
IR written to: D:\TEMP\rust-demo\course_full.ir
Assembly written to: D:\TEMP\rust-demo\course_full.s
Executable written to: D:\TEMP\rust-demo\course_full.exe
Compilation succeeded: lexer, parser, semantic analysis, IR and target code
completed.

PS> & D:\TEMP\rust-demo\course_full.exe
PS> $LASTEXITCODE
42
```

八、测试设计与实验结果

8.1 分层测试策略

测试不只统计样例文件，而是按编译阶段组织：文件正例/负例验证整体行为；词法边界验证关键字和最长匹配；AST 断言验证文法结构；IR 断言验证控制流和表达式翻译；内存语义用例验证不可变性、借用、越界和返回路径；管线专项检查按需 Lexer 与函数回调；最后由 GCC 汇编执行综合程序。

层次	代表检查	预期
文件正例	minimum、v1_ir、v2_compare、course_full	完整通过
文件负例	err_lex、err_parse、*_error	在正确阶段拒绝
词法	if123、if=123、->、..	边界与最长匹配正确
AST	p1/p2/p3 节点	节点类型和层次正确
IR	if/while/for/call/if expr/loop expr	标签、临时值和跳转正确
语义	mut、借用、数组/元组、return	合法放行、非法拦截
管线	lexer_on_demand_and_function_codegen_hook	未预扫描且回调次数正确
目标	assemble_and_execute	GCC 成功且退出码 42

8.2 44 组回归结果

本轮实际执行 `python run_regression.py`，所有检查均输出 [PASS]，末尾为 Regression passed。44 组由 12 个正确源文件、4 个错误源文件、3 个词法检查、3 个 AST 检查、6 个 IR 检查、14 个语义检查、1 个管线检查和 1 个目标执行检查组成。这里的数字直接来自最终代码运行结果，没有沿用旧报告统计。

类别	数量	结果
正确源文件	12	12/12 通过
错误源文件	4	4/4 被正确拒绝
词法专项	3	3/3 通过
AST 专项	3	3/3 通过
IR 专项	6	6/6 通过

类别	数量	结果
语义专项	14	14/14 通过
按需管线	1	1/1 通过
汇编执行	1	1/1 通过，退出码 42
合计	44	44/44 通过

8.3 综合样例覆盖

`course_full.rs` 定义 `add`、`classify` 和 `main` 三个函数，覆盖多参数调用、函数尾表达式、`if/else-if/else`、范围 `for` 与 `continue`、数组声明/初始化/下标、元组与字段写入、可变引用和解引用、`while`、`loop` 表达式、`if` 表达式、块表达式以及嵌套算术。最终表达式调用 `add`，理论值和机器执行值均为 42。

8.4 关键缺陷的回归证据

- **for continue:** IR 必须先跳到自增标签，再返回条件标签。
- **重影:** 同名声明必须生成不同 `vN_name`，目标栈槽不再覆盖。
- **借用互斥:** 存在不可变借用时拒绝 `&mut`；可变借用期间拒绝直接访问原变量。
- **不可变首次初始化:** `let a:i32; a=1` 合法，之后 `a=2` 非法。
- **返回路径:** 声明 `i32` 的函数若存在可达无返回路径必须拒绝。
- **数组边界:** 常量越界在语义阶段拒绝，动态越界在目标层执行 `ud2`。

8.5 源码入口与打包程序交叉验证

同一个 `course_full.rs` 分别通过 `python run.py` 和 `dist/rust_like_compiler.exe` 编译，二者均产生 2779 字节左右的 IR、6715 字节左右的汇编和 121976 字节左右的目标程序；两份目标程序执行退出码均为 42。交叉验证降低了“源码可运行但打包缺模块”或“打包程序使用旧代码”的风险。

8.6 测试局限

回归覆盖课程文法和已知边界，但不等同形式证明。未覆盖随机生成程序、超大输入性能、所有 GCC 版本、Linux System V ABI 或真正 Rust 生命周期。后续可引入基于文法的属性测试、IR 解释器对照、差分执行和控制流图覆盖率。

九、问题解决与 AI 辅助历程

9.1 从分离流水线改为 Parser 主控

旧流程先 `Lexer.tokenize` 得到完整列表，再构造 `Parser`，虽然模块清楚，但不符合附件“词法分析作为子程序，需要时由语法分析调用”的硬约束。改造时保留 `tokenize` 作为调试兼容接口，正式 `Compiler` 传入 `Lexer`；`Parser` 通过 `_ensure_token` 惰性填充缓存。专项测试使用计数 `Lexer` 证明构造 `Parser` 后不会立即扫描全文件，并核对函数回调次数。

9.2 控制流与重影缺陷

AI 辅助审查首先指出 `for` 的 `continue` 目标不能与 `while` 相同。根据降低后的控制流逐条推演，确认原 `IR` 会跳过自增，于是新增 `increment` 标签。进一步检查目标栈布局时发现 `AST`/语义允许重影，而 `IR` 名称仍复用源变量名，造成后绑定覆盖前栈槽；通过作用域重命名器分配唯一内部名解决。两个缺陷都增加了回归断言，而不是只修改实现。

9.3 借用、返回与边界规则补强

初版语义分析只检查部分 `mut` 规则，缺少多不可变借用、可变借用冲突、借用释放、不可变首次初始化和完整返回路径。根据附件 6.x 语义说明和实际 `AST`，补充符号状态与控制流返回判断。数组常量边界在语义阶段处理，动态边界在汇编后端处理，形成编译期与运行期分工。

9.4 目标后端中的工程问题

目标后端需要同时考虑教学类型、指针宽度、栈对齐、`shadow space`、参数位置和复合对象存储。为降低复杂度，物理槽统一为 64 位，数组/元组变量保存 `backing` 首地址；函数标签增加函数名前缀；所有 `return` 汇聚到统一尾声。遇到未知 `IR` 操作时显式报错，避免生成表面可汇编但语义错误的代码。

9.5 AI 工具使用披露

本次课程设计于 2026 年 7 月 22 日使用 `OpenAI Codex` 辅助读取课程要求、梳理旧代码差距、提出实现候选、补充测试、重写 `README`，并生成报告与 `PPT` 初稿。AI 参与了代码建议和文档组织，因此本项目不把这些环节描述为完全人工完成。

对 AI 产出采用可复核审查：第一，所有实现必须落到现有数据模型，不允许为展示效果虚构功能；第二，运行 `compileall` 和 44 组回归；第三，用 GCC 生成并实际执行目标程序；第四，使用源码入口与打包 EXE 交叉验证；第五，报告中的数字、文件名、限制和代码片段以最终工作区为准；第六，Word/PPT 均逐页渲染检查。

9.6 对 AI 辅助的反思

AI 的优势是能快速跨越文法、IR、ABI、测试和文档多个层次，尤其擅长发现接口间的不一致；风险是容易把“生成了代码”误当成“满足了语义”，或给出超出课程范围的复杂设计。本项目最重要的经验不是接受更多生成内容，而是把每项建议转换为可观察的不变量：扫描次数、回调时机、四元式序列、栈槽唯一性、退出码和错误阶段。只有这些证据成立，AI 建议才被保留。

十、总结与展望

10.1 工作总结

本项目完成了从类 Rust 源程序到 Token、AST、语义校验、四元式、x86-64 汇编和 Windows 可执行程序的完整链路。与旧作业相比，最核心的变化是 Parser 按需驱动 Lexer，并在函数归约点生成 IR；同时新增目标后端、文件输出、独立编译器和端到端执行验证。

规则层面覆盖附件 0.1–9.3，包含重影、控制流表达式、引用借用、数组元组等扩展；质量层面修复了 continue、自增、重影栈槽、借用互斥、首次初始化、返回路径和越界等问题。44 组回归全部通过，综合程序退出码 42，为课程要求中的 IR 与可执行目标代码提供了可复现证据。

10.2 后续工作

- 构建控制流图和 SSA，增加常量折叠、死代码消除和寄存器分配。
- 把借用检查扩展为非词法生命周期，并加入移动语义和所有权转移。
- 支持更多基础类型、字符串、结构体和堆对象。
- 增加 Linux System V ABI 后端和跨平台工具链检测。
- 引入文法随机生成、IR 解释器差分测试和覆盖率统计。
- 把错误恢复加入 Parser，使一次编译报告多个独立错误。

10.3 设计体会

课程设计使抽象概念转化为可执行约束：最长匹配体现在 Token 边界，左递归消除体现在循环式解析，语法制导翻译体现在 FunctionDecl 回调，符号表体现在作用域栈，控制流体现在标签与跳转，调用约定最终体现在真实寄存器与栈。只有当目标程序运行并给出预期退出码时，这些阶段才真正构成一条编译链。

参考资料

- [1] Aho A V, Lam M S, Sethi R, Ullman J D. Compilers: Principles, Techniques, and Tools (2nd Edition). Addison-Wesley, 2006.
- [2] Appel A W. Modern Compiler Implementation in C. Cambridge University Press, 1998.
- [3] The Rust Project Developers. The Rust Reference. <https://doc.rust-lang.org/reference/>.
- [4] Klabnik S, Nichols C. The Rust Programming Language (2nd Edition). No Starch Press, 2023.
- [5] Microsoft. x64 calling convention. <https://learn.microsoft.com/cpp/build/x64-calling-convention>.
- [6] Free Software Foundation. Using the GNU Compiler Collection. <https://gcc.gnu.org/onlinedocs/>.
- [7] PyInstaller Development Team. PyInstaller Manual. <https://pyinstaller.org/en/stable/>.
- [8] 同济大学计算机系. 《编译原理课程设计 2026（类 Rust 编译器实现）》课程附件.

附录 A：核心文法摘要

```
Program      ::= FunctionDecl* '#'? EOF
FunctionDecl ::= 'fn' ID '(' Params? ')' ('->' Type)? Block
Type         ::= 'i32' | '&' 'mut'? Type
              | '[' Type ';' NUM ']'
              | '(' ' ' ')' | '(' Type ',' (Type (',' Type)*)? ')'
Statement    ::= ';' | Let | Assign | Return | If | While | For | Loop
              | 'break' Expression? ';' | 'continue' ';' | Expression ';'
Expression   ::= comparison
comparison   ::= additive (compare_op additive)*
additive     ::= term (('+'|'-') term)*
term         ::= factor (('*'|'/') factor)*
factor       ::= NUM | ID | Call | Borrow | Deref | Array | Tuple
              | BlockExpr | IfExpr | LoopExpr | factor '[' Expression ']'
              | factor '.' NUM
```

完整实现以 `src/parser.py` 为准。附件中的个别排版疑点（7.5 未定义、元组示例分隔符等）按正文语义意图和测试一致性处理。

附录 B：复现实验命令

```
cd analyzer
python -m compileall -q src run.py run_regression.py
python run_regression.py

python run.py examples/course_full.rs --no-tokens --no-ast --assemble
& examples/build/course_full.exe
$LASTEXITCODE

dist/rust_like_compiler.exe examples/course_full.rs `
--no-tokens --no-ast --no-ir --output-dir build/exe-check --assemble
```

若未安装 GCC，回归程序的目标执行项会显示 [SKIP]，其余前端检查仍可运行；申请优秀档验收时应在具备 GCC 的环境中完成完整目标代码验证。